IMT Nord Europe
École Mines-Télécom
IMT-Université de Lille

RAPPORT DE TRAVAUX PRATIQUES

PROJET IoT TEMPHUMIX

Simulation Arduino et supervision par IHM

Étudiant

REN Ruilang

Résumé et organisation du rapport

Ce rapport présente le système IoT TempHumiX, depuis les simulations sous SimulIDE jusqu'au boîtier Arduino UNO R4 WiFi et à son interface de supervision Python. Le projet associe mesure de température et d'humidité, affichage local, synchronisation de l'heure, communication MQTT, archivage SQLite et consultation des courbes.

Les essais vérifient les fonctions du boîtier et le dialogue MQTT. La partie IHM décrit les programmes console, l'interface PySide6, les commandes et l'historique. Elle comprend une notice de fonctionnement ainsi qu'un bilan des difficultés et des améliorations, appuyés par les captures des séances d'octobre et du 3 novembre 2026.

Mots-clés : SimulIDE ; Arduino UNO R4 WiFi ; MQTT ; PySide6 ; SQLite ; Matplotlib.

Sommaire

Partie	Contenu	Page
Introduction	Objectifs, environnement et méthode	3
3.1	Exercices élémentaires sous SimulIDE	4
3.2	Simulation intégrée de TempHumiX	9
6.1.1 et 6.1.2	Acquisition et horloge RTC	10
6.1.3	WiFi et synchronisation NTP	12
6.1.4	Publication MQTT	14
6.1.5	Abonnements et commande à distance	16
6.1.6	Optimisation des publications	18
6.2.1	Analyse des échanges avec Wireshark	20
6.2.2	Retain, QoS, filtres et supervision	21
IHM 3	Préparation et communication MQTT	26
IHM 3.6 et 4	Interface, acquisition et commandes	27
IHM 5 et 6	SQLite et courbes historiques	29
IHM bilan	Difficultés et améliorations	31
Bilan	Validation, difficultés et versionnement	32
Conclusion	Conclusion et ressources	33

Introduction et environnement expérimental

Objectifs du projet

L'objectif est de construire une chaîne complète allant de la mesure physique à sa consultation à distance, puis de permettre la commande du capteur depuis un client MQTT. La simulation sert à vérifier les entrées et sorties avant leur intégration au matériel réel. Les essais réseau vérifient ensuite le transport des informations et le comportement du protocole.

Élément	Fonction
SimulIDE et Arduino UNO simulé	Développement et validation des fonctions locales.
Arduino UNO R4 WiFi	Exécution du programme, RTC et connexion au réseau.
DHT22 et LCD I2C	Acquisition de la température et de l'humidité ; affichage local.
LED de vie et bargraphe	Signalisation du fonctionnement et commande de dix segments.
MQTTX et Wireshark	Essais de publication et d'abonnement ; analyse des paquets.
Git et GitHub	Historique des programmes et des schémas de simulation.

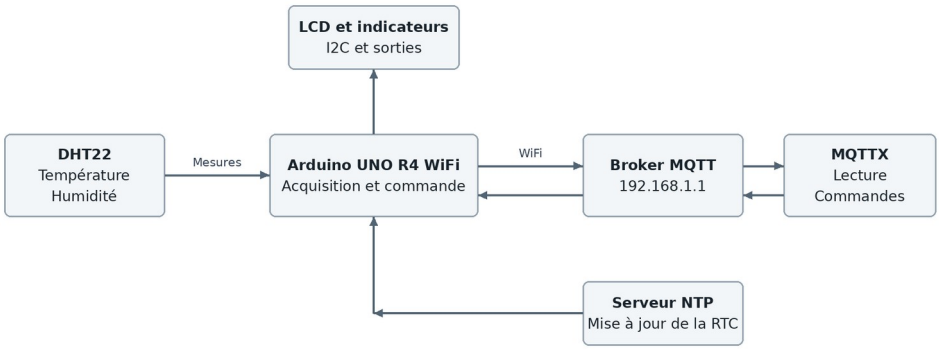


Figure 1 - Architecture fonctionnelle du prototype et de ses services réseau.

Méthode de validation

Chaque étape est décrite par son objectif, sa réalisation et son résultat. Les six captures de simulation, les photographies du boîtier et les extraits vidéo constituent les illustrations expérimentales. Pour les essais de protocole, les observations sont complétées par les validations consignées pendant la séance du 21 octobre 2026. Les valeurs envoyées manuellement dans MQTTX sont distinguées des mesures du DHT22.

3.1 Exercices élémentaires sous SimulIDE

Clignotement d'une LED

Le premier montage vérifie une sortie numérique. La LED est reliée à D2 par une résistance de 100 Ω, avec retour à la masse. Le programme alterne les états logiques pour produire un clignotement. La capture correspond à un instant où la LED est allumée ; la périodicité s'observe pendant l'exécution.

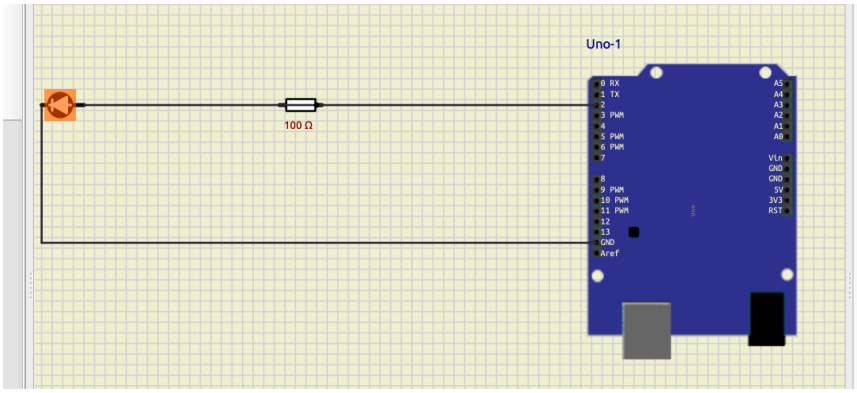


Figure 2 - Montage de la LED sur D2. La résistance de 100 Ω est celle du schéma simulé.

Chenillard sur un bargraphe

Le deuxième exercice étend cette commande à dix sorties, de D3 à D12. Le programme parcourt les segments avec une temporisation de 100 ms entre deux positions. L'activation successive permet de vérifier le câblage et l'ordre de parcours. Sur la capture, un seul segment est allumé.

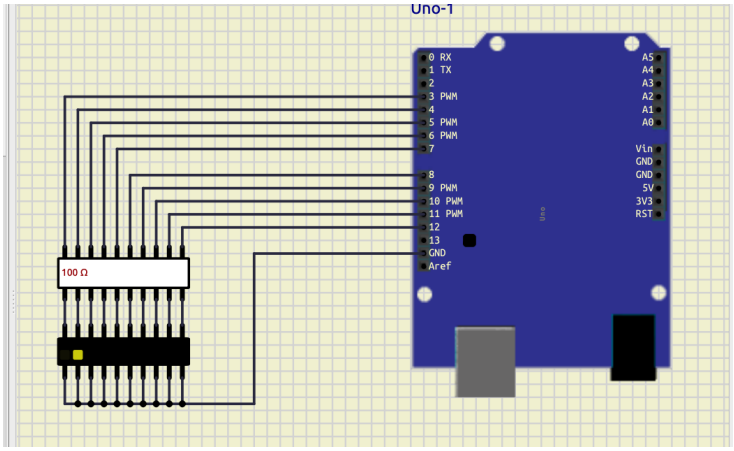


Figure 3 - Chenillard simulé sur dix LED, reliées à D3–D12 par un réseau de résistances de 100 Ω.

Résultat : les sorties peuvent être pilotées séparément. Ces essais préparent le contrôle des indicateurs du système intégré.

3.1 Programmation des sorties numériques

Les premiers programmes isolent les sorties pour vérifier les affectations et les temporisations. Leur logique est volontairement simple ; les attentes bloquantes seront ensuite remplacées par une gestion fondée sur millis() dans le système intégré.

Code 1 - Commande de la LED sur D2

```
#define PIN_LED 2
void setup() {
  pinMode(PIN_LED, OUTPUT);
}
void loop() {
  digitalWrite(PIN_LED, HIGH);
  delay(500);
  digitalWrite(PIN_LED, LOW);
  delay(500);
}
```

La broche est d'abord configurée en sortie. Chaque état dure 500 ms : le cycle complet dure une seconde, soit 1 Hz. Il s'agit du montage élémentaire ; la LED de vie du système intégré est ensuite réglée à 2 Hz.

Code 2 - Parcours séquentiel du bargraphe

```
const int LED_START = 3;
const int LED_COUNT = 10;
void loop() {
  for (int i = 0; i < LED_COUNT; i++) {
    digitalWrite(LED_START + i, HIGH);
    delay(100);
    digitalWrite(LED_START + i, LOW);
  }
}
```

L'index i sélectionne les broches D3 à D12. La LED courante est éteinte avant le passage à la suivante : un seul segment reste actif à chaque étape. Les dix positions donnent un parcours d'environ une seconde, hors temps d'exécution des instructions.

3.1 Acquisition du DHT22 et affichage I2C

Lecture du capteur et commande des LED

Le DHT22 simulé est raccordé à A0. Les valeurs imposées sont 22,5 °C et 68,5 % d'humidité relative. Deux sorties lumineuses complètent le montage ; la capture montre la LED inférieure allumée. Cet exercice associe la lecture d'une entrée à une décision de commande dans le programme.

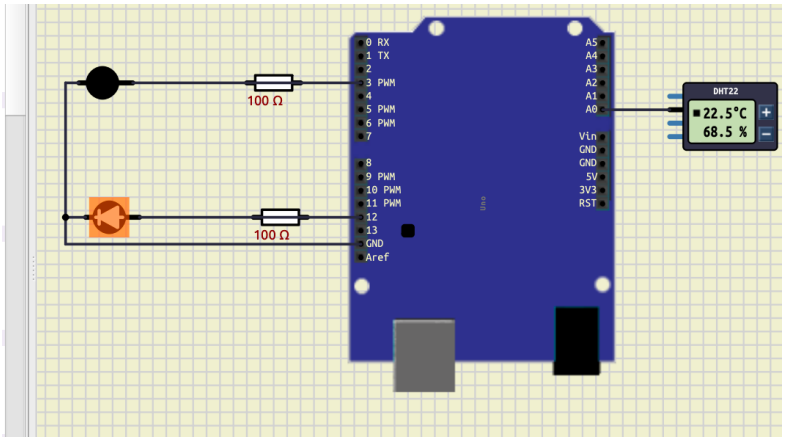


Figure 4 - DHT22 réglé à 22,5 °C et 68,5 % ; état des deux indicateurs du montage.

Initialisation du LCD I2C

L'afficheur simulé utilise le contrôleur AIP31068. SDA est reliée à A4 et SCL à A5 sur l'UNO simulé. L'écriture de HELLO sur la première ligne et de WORLD sur la seconde valide l'initialisation, la communication I2C et le positionnement du texte.

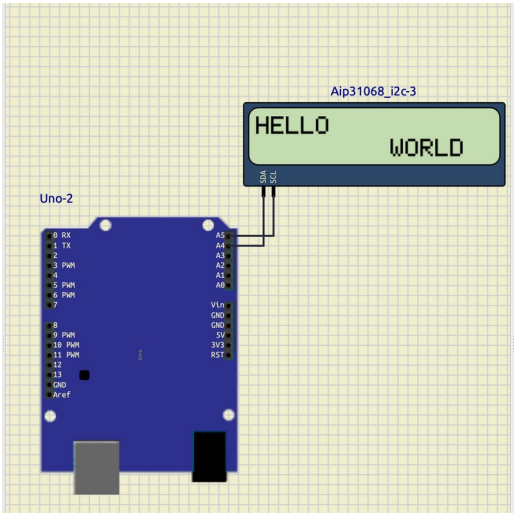


Figure 5 - Essai du LCD I2C avec affichage de HELLO et de WORLD sur deux lignes.

3.1 Association du LCD et du DHT22

Le programme réunit ensuite le capteur et l'afficheur. La première ligne identifie le projet avec le texte IMT NE - PPI ; la seconde restitue la température et l'humidité. La capture permet une comparaison directe entre les valeurs du composant simulé et celles présentées à l'utilisateur.

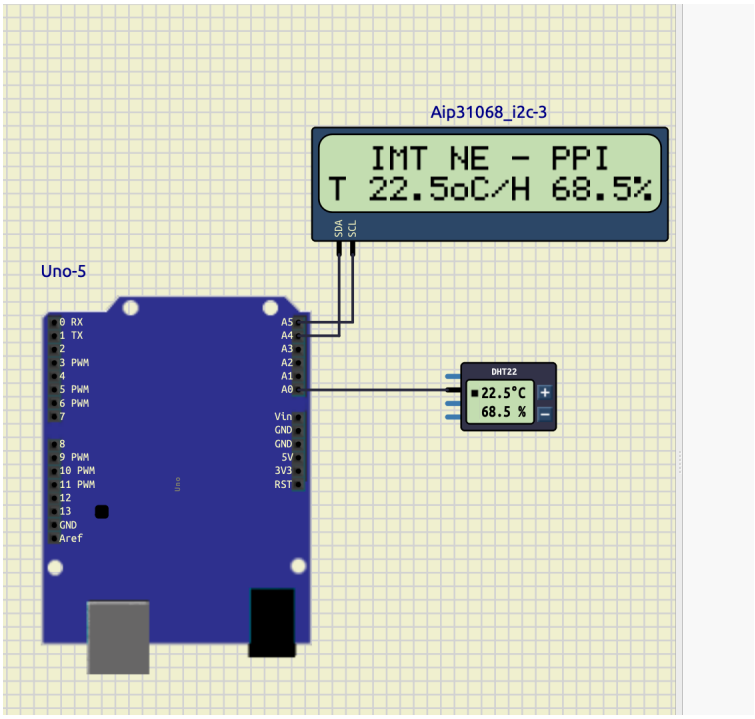


Figure 6 - Concordance entre le DHT22 simulé et le LCD : 22,5 °C et 68,5 %.

Liaison	Affectation dans la simulation
DHT22 DATA	A0
LCD SDA	A4
LCD SCL	A5
Température de l'essai	22,5 °C
Humidité relative de l'essai	68,5 %

Résultat : les deux valeurs affichées correspondent aux valeurs réglées sur le DHT22. Le test valide le transfert et la mise en forme des données dans le simulateur. La précision du capteur physique sera une question distincte lors du passage au boîtier réel.

3.1 Traitement des mesures et affichage LCD

L'acquisition est d'abord associée à une règle de commande, puis à une restitution sur le LCD. Les extraits ci-dessous relient les états visibles dans les captures aux comparaisons et aux opérations d'écriture du programme.

Code 3 - Choix des indicateurs selon le seuil de température

```
if (isnan(temperature)) {
  Serial.println("Erreur DHT22");
  digitalWrite(LED_VERTE, LOW);
  digitalWrite(LED_ROUGE, LOW);
}
else if (temperature < 20.0) {
  digitalWrite(LED_VERTE, HIGH);
  digitalWrite(LED_ROUGE, LOW);
}
else {
  digitalWrite(LED_VERTE, LOW);
  digitalWrite(LED_ROUGE, HIGH);
}
```

Le programme distingue une erreur, une température inférieure à 20 °C et une température supérieure ou égale à ce seuil. Avec 22,5 °C, la dernière branche s'exécute. En cas d'erreur de lecture, les deux sorties sont éteintes.

Code 4 - Mise en forme des mesures sur la seconde ligne du LCD

```
lcd.setCursor(0, 1);
lcd.print("                ");
lcd.setCursor(0, 1);
lcd.print("T ");
lcd.print(temperature, 1);
lcd.print("oC/H ");
lcd.print(humidite, 1);
lcd.print("%");
```

Les seize espaces effacent l'ancienne valeur sans modifier la première ligne. Le curseur revient ensuite au début de la ligne 1, car les indices commencent à zéro. Le paramètre 1 de print conserve une décimale. Le texte oC est la notation utilisée dans le sketch de simulation.

3.2 Simulation intégrée de TempHumiX

Le montage final associe le DHT22, le LCD I2C, le bargraphe, une LED de vie et le terminal série. Le programme ajoute une horloge logicielle et restitue les mesures sur plusieurs sorties. Cette intégration prépare l'organisation du programme embarqué, avant l'ajout du WiFi et de MQTT.

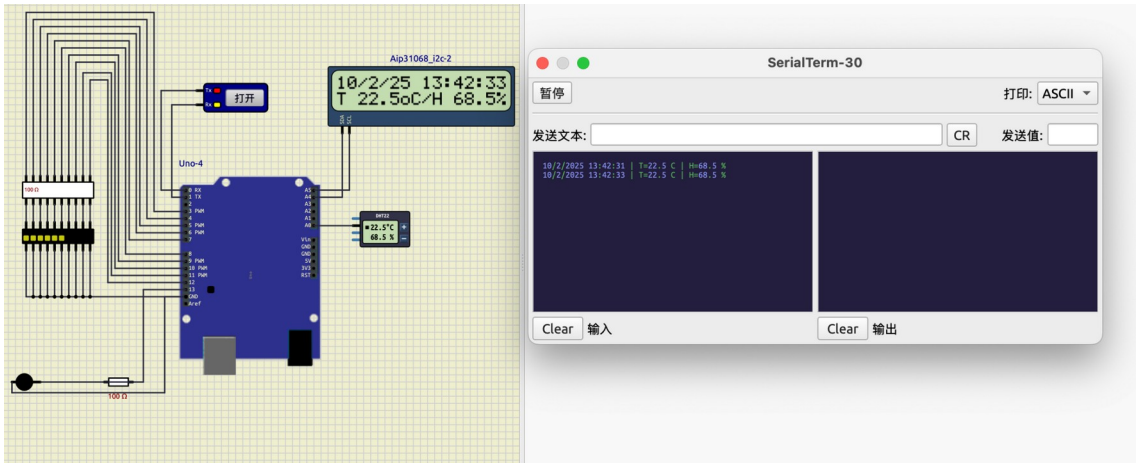


Figure 7 - Vue complète de TempHumiX dans SimulIDE avec affichage local, bargraphe et terminal série.

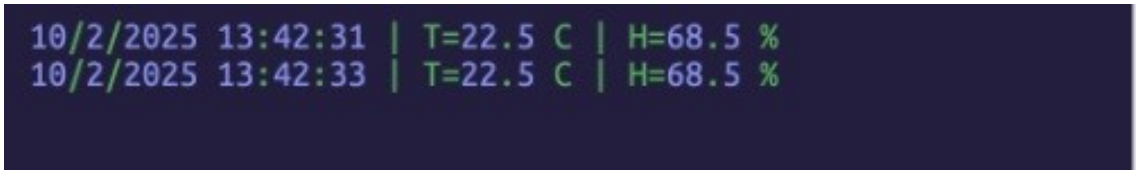


Figure 8 - Détail du terminal : deux mesures successives à 13:42:31 et 13:42:33, pour 22,5 °C et 68,5 %.

Résultat de l'intégration

Le LCD et le terminal présentent 22,5 °C et 68,5 %, avec deux secondes entre les mesures. Les six segments allumés correspondent à l'arrondi de 22,5 / 4. La date provient de l'horloge de démonstration.

Code 5 - Temporisation de la LED sans attente de 250 ms

```
unsigned long maintenant = millis();
if (maintenant - dernierClignotement >= 250) {
  dernierClignotement += 250;
  etatLed = !etatLed;
  digitalWrite(LED_VIE, etatLed);
}
```

Un changement d'état toutes les 250 ms produit un cycle de 500 ms, soit 2 Hz. Le test de durée laisse la boucle traiter l'horloge et le capteur entre deux changements.

L'horloge logicielle utilise des mois de 30 jours. Sur l'UNO R4 WiFi, elle est remplacée par la RTC et les bibliothèques sont adaptées. Le terminal simulé vérifie la fonction série, sans reproduire un étage RS232.

6.1.1 et 6.1.2 Acquisition et horloge RTC

6.1.1 Acquisition et affichage local

Le premier programme sur le boîtier lit le DHT22 toutes les 2 000 ms et affiche les valeurs sur le LCD. La LED de vie indique l'exécution. Une correction de son affectation vers D2 a rétabli son fonctionnement pendant les essais. La photographie montre une température de 27,3 °C et une humidité relative de 45,3 %.

Code 6 - Lecture du DHT22 et rejet des valeurs invalides

```
float temperature = dht.readTemperature();
float humidite = dht.readHumidity();
if (isnan(temperature) || isnan(humidite)) {
  lcd.setCursor(0, 1);
  lcd.print("Erreur DHT22    ");
  Serial.println("Erreur DHT22 (verifier broche A0)");
  return;
}
```

Une valeur NaN signale une lecture inexploitable. Le retour immédiat évite de l'afficher comme une mesure valide ; l'erreur reste visible sur le LCD et dans le moniteur série.

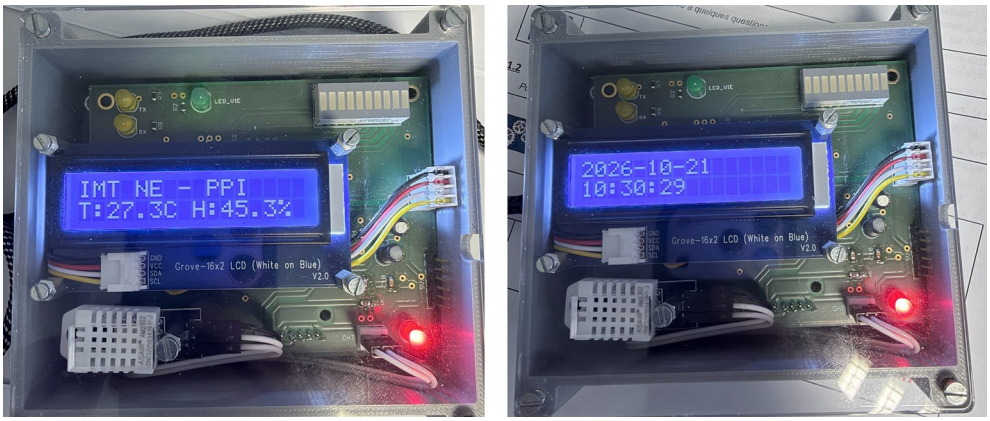


Figure 9 - À gauche, affichage local des mesures ; à droite, lecture de la date et de l'heure par la RTC.

6.1.2 Gestion de la RTC

La deuxième étape ajoute la lecture de l'horloge temps réel de l'UNO R4. L'affichage de la date et de l'heure est actualisé chaque seconde. La photographie présente le 21 octobre 2026 à 10:30:29. L'essai valide la lecture de la RTC et le formatage sur les deux lignes du LCD.

Organisation des tâches

Les fonctions suivent des rythmes distincts. Les versions réseau utilisent ensuite des tests avec millis() pour maintenir le traitement MQTT entre deux mesures.

6.1.2 Initialisation et lecture de la RTC

Le test RTC initialise une date de démonstration, puis relit l'horloge de la carte. Cette étape vérifie le matériel et le format d'affichage avant l'introduction du serveur NTP dans l'exercice suivant.

Code 7 - Initialisation de l'horloge avec une date de test

```
if (!RTC.begin()) {
  lcd.print("Erreur RTC");
  while (true) {}
}
RTCTime startTime(
  21, Month::OCTOBER, 2026,
  10, 30, 0,
  DayOfWeek::WEDNESDAY,
  SaveLight::SAVING_TIME_ACTIVE
);
RTC.setTime(startTime);
```

RTC.begin() vérifie la disponibilité de l'horloge. Le constructeur rassemble la date, l'heure et les indicateurs du test. RTC.setTime() charge cette valeur à chaque démarrage du sketch : ce programme constitue un essai initial et non une synchronisation réseau.

Code 8 - Lecture de la date et de l'heure sur deux lignes

```
RTCTime currentTime;
if (RTC.getTime(currentTime)) {
  String dateTime = currentTime.toString();
  lcd.setCursor(0, 0);
  lcd.print(dateTime.substring(0, 10));
  lcd.setCursor(0, 1);
  lcd.print(dateTime.substring(11, 19));
  Serial.println(dateTime);
}
delay(1000);
```

Le programme utilise le format retourné par toString() : les dix premiers caractères représentent la date et les huit caractères suivants représentent l'heure. La lecture est répétée environ chaque seconde. Le même texte est envoyé au moniteur série pour comparer les deux restitutions.

6.1.3 Connexion WiFi et synchronisation NTP

La carte se connecte au réseau IoT IMT Nord Europe, récupère une adresse IP et interroge un serveur NTP. L'heure obtenue est utilisée pour mettre à jour la RTC. La vidéo de cet exercice montre successivement le démarrage, la version avec l'adresse IP, puis la date et les mesures.



Figure 10 - Extraits de la vidéo 6.1.3 à 0 s, 2 s et 8 s : démarrage, réseau, puis affichage horodaté.

Paramètre	Valeur observée ou utilisée
Réseau WiFi	IoT IMT Nord Europe
Adresse du boîtier	192.168.1.108
Serveur essayé initialement	10.12.130.1 : requêtes NTP sans réponse exploitable.
Serveur utilisé avec succès	192.168.1.254
Services	NTP sur UDP ; MQTT sur TCP, port 1883.

Diagnostic et résultat

Le serveur initial répondait au ping depuis le Mac, mais la synchronisation NTP échouait depuis le programme. Une réponse ICMP ne suffit pas à établir la disponibilité du service NTP. Le changement de serveur vers 192.168.1.254 a permis la synchronisation, confirmée pendant la séance.

À 8 s, le boîtier affiche le 21/10 à 14:06:48, avec 26,1 °C et 53,1 %. La vidéo illustre la séquence de fonctionnement et l'affichage horodaté ; la validation de la réponse NTP provient également du diagnostic réalisé pendant l'essai.

6.1.3 Gestion de la connexion et mise à l'heure

La synchronisation dépend de l'état du réseau et de la validité de la réponse reçue. Les extraits proviennent du sketch de l'exercice 6.1.3 ; les messages de diagnostic secondaires sont omis pour conserver la logique principale.

Code 9 - Validation du WiFi avant le démarrage de NTP

```
bool connexionOK = wifiOK();
if (connexionOK && !wifiConnecte) {
  wifiConnecte = true;
  Serial.println("WiFi connecte !");
  Serial.print("Adresse IP : ");
  Serial.println(WiFi.localIP());
  ntp.begin();
  ntpDemarre = true;
  derniereTentativeNTP = millis() - 15000UL;
}
```

Le traitement détecte le passage vers un état connecté. NTP est démarré après cette transition. La soustraction de 15 000 ms autorise la première tentative sans attendre un intervalle supplémentaire ; les calculs utilisent des compteurs non signés.

Code 10 - Validation du timestamp et transfert de l'heure UTC vers la RTC

```
if (ntp.forceUpdate()) {
  unsigned long epoch = ntp.getEpochTime();
  if (epoch < 1700000000UL) {
    Serial.println("Date NTP invalide");
    return;
  }
  RTCTime heureUTC((time_t)epoch);
  if (RTC.setTime(heureUTC)) {
    if (!heureSynchronisee) {
      debutAccueil = millis();
    }
    heureSynchronisee = true;
    Serial.println("NTP OK !");
  }
}
```

Le seuil écarte une date manifestement trop ancienne ; il ne garantit pas à lui seul l'exactitude de l'heure reçue. La RTC conserve l'heure UTC. Une fonction séparée applique le décalage français pour l'affichage. Le sketch prévoit un nouvel essai après 15 s en cas d'échec, puis un intervalle de 15 min après succès.

6.1.4 Publication des informations par MQTT

Après la connexion WiFi, le programme ouvre une session vers le broker 192.168.1.1 sur le port 1883. Il publie la version, la température, l'humidité et la date. L'identifiant 44666 distingue ce capteur des autres équipements. MQTTX s'abonne à /iot/# pour recevoir ces informations.

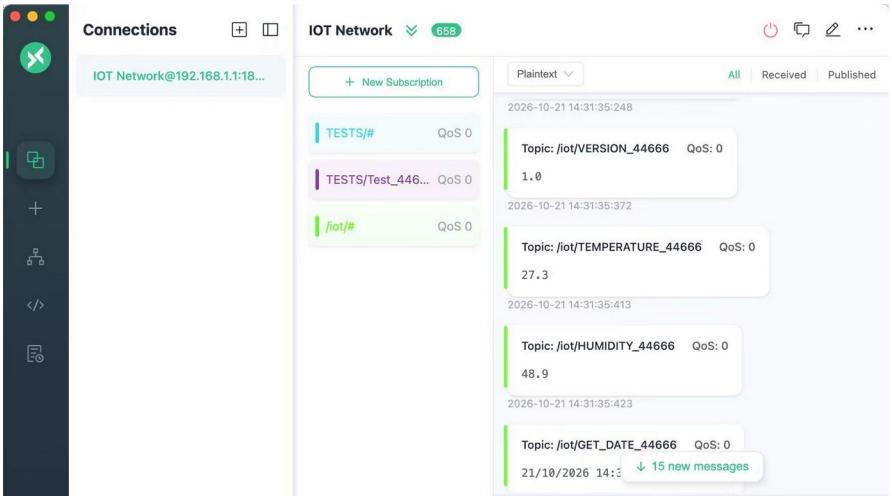


Figure 11 - Réception dans MQTTX des quatre informations publiées par le capteur 44666.

Topic	Contenu visible
/iot/VERSION_44666	1.0
/iot/TEMPERATURE_44666	27.3
/iot/HUMIDITY_44666	48.9
/iot/GET_DATE_44666	Date et heure de la mesure.

Résultat : MQTTX reçoit les quatre informations, affichées avec QoS 0. Les valeurs varient entre les prises de vue. Le préfixe, la casse et l'identifiant des topics doivent correspondre exactement aux abonnements.

Code 11 - Fonction de publication réutilisée dans la version finale

```
bool publierChaine(const char *topic, const char *message) {
  if (!mqttClient.connected()) return false;
  if (!mqttClient.beginMessage(topic)) return false;
  mqttClient.print(message);
  if (!mqttClient.endMessage()) return false;
  return true;
}
```

La fonction vérifie la connexion, prépare le topic puis écrit la charge utile. Le booléen indique le succès des appels de la bibliothèque ; il ne constitue pas un acquittement MQTT du broker en QoS 0.

6.1.4 Entretien de la session et préparation des données

La publication nécessite une session disponible et un contenu correctement formaté. La version finale réutilise ces fonctions pour gérer les pertes de connexion et produire des charges utiles cohérentes avec les valeurs affichées.

Code 12 - Entretien de MQTT et tentative de reconnexion

```
void gererMQTT() {
  if (!wifiConnecte) return;
  if (mqttClient.connected()) {
    mqttClient.poll();
  } else if (millis() - dernierMQTT >= 10000UL) {
    dernierMQTT = millis();
    setup_mqtt();
  }
}
```

poll() traite les échanges du client et permet le déclenchement de la fonction de réception enregistrée. En cas de perte de session, setup_mqtt() est rappelée au plus toutes les dix secondes. L'état du WiFi constitue une condition préalable ; le délai de reprise évite une succession immédiate de tentatives.

Code 13 - Formatage des valeurs et publication des trois informations

```
char valeurTemp[16];
char valeurHum[16];
dtostrf(temperature, 0, 1, valeurTemp);
dtostrf(humidite, 0, 1, valeurHum);
bool okTemp = publierChaine(TOPIC_TEMP, valeurTemp);
bool okHum = publierChaine(TOPIC_HUM, valeurHum);
bool okDate = publierChaine(TOPIC_DATE, dateHeure);
```

Les valeurs sont converties en texte avec une décimale avant émission. dateHeure est préparée auparavant à partir de la RTC. Les trois booléens sont conservés séparément pour vérifier le résultat de chaque appel. Ces publications restent trois messages distincts, et non une transaction atomique.

6.1.5 Abonnements et commande à distance

Le capteur reçoit ensuite les topics COMMAND, BARGRAPH et BLINK associés à 44666. MQTTX publie les consignes ; le programme les décode et agit sur les mesures ou les indicateurs. La vidéo fournie pour 6.1.5 documente cette commande à distance.

Topic	Rôle
/iot/COMMAND_44666	Sélection d'une action par un code numérique.
/iot/BARGRAPH_44666	Écriture du masque des dix segments, de 0 à 1023.
/iot/BLINK_44666	Réglage du clignotement personnalisé.

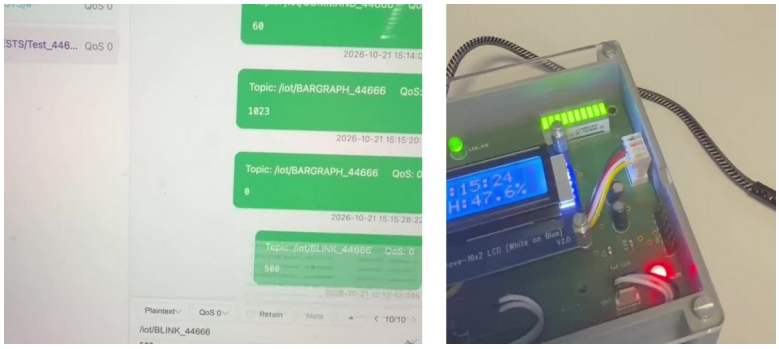


Figure 12 - Extraits de la vidéo 6.1.5 : publication de 1023 dans MQTTX et activation des dix segments du boîtier.

Code COMMAND	Action
10	Forcer la synchronisation de la date et de l'heure par NTP.
20 / 30	Démarrer / arrêter les mesures.
40 / 50	Choisir un clignotement lent / rapide.
60	Appliquer le clignotement personnalisé réglé par BLINK.

Résultat : les commandes ont été testées pendant la séance. La vidéo relie la publication de 1023 dans MQTTX à l'allumage des dix segments.

Code 14 - Décodage du masque binaire du bargraphe

```
for (int i = 0; i < BAR_COUNT; i++) {
  bool actif = (valeur & (1 << i)) != 0;
  digitalWrite(BAR_START + i, actif ? HIGH : LOW);
}
```

Chaque bit pilote un segment. Avec BAR_COUNT = 10 et BAR_START = 3, 1023 allume D3 à D12 ; 0 les éteint. Le programme vérifie au préalable que la consigne reste comprise entre 0 et 1023.

6.1.5 Abonnements et interprétation des commandes

La réception comporte deux opérations : déclarer les filtres d'abonnement, puis associer le contenu du message à une action. Les fragments montrent les contrôles de souscription et la logique de démarrage ou d'arrêt des mesures.

Code 15 - Souscription aux trois topics du boîtier

```
bool s1 = mqttClient.subscribe(TOPIC_COMMAND, 0);
bool s2 = mqttClient.subscribe(TOPIC_BARGRAPH, 0);
bool s3 = mqttClient.subscribe(TOPIC_BLINK, 0);
if (!s1 || !s2 || !s3) {
  Serial.println("Erreur souscription MQTT");
  mqttClient.stop();
  return -11;
}
```

Les trois souscriptions demandent QoS 0. Si un appel échoue, la session est fermée et la fonction retourne un code d'erreur. Le programme ne conserve ainsi pas volontairement une connexion où seule une partie des topics de commande serait active.

Code 16 - Lecture du message puis traitement de START et STOP

```
String topic = mqttClient.messageTopic();
String message = "";
while (mqttClient.available()) {
  char c = (char)mqttClient.read();
  if (message.length() < 20) {
    message += c;
  }
}
message.trim();
long valeur = message.toInt();
if (topic == TOPIC_COMMAND) {
  switch (valeur) {
    case 20:
      mesuresActives = true;
      derniereMesure = millis() - 2000UL;
      break;
    case 30:
      mesuresActives = false;
      publicationMesuresEnAttente = false;
      break;
  }
}
```

Le fragment conserve deux branches de COMMAND. Dans le fichier complet, la taille et les chiffres sont validés avant toInt(). START autorise une acquisition immédiate. STOP désactive les mesures et annule une publication en attente. Les autres codes du switch et les diagnostics sont omis.

6.1.6 Optimisation des publications

La dernière modification compare chaque mesure valide aux dernières valeurs publiées avec succès. Une première publication est également autorisée. Si la température ou l'humidité diffère, le programme demande l'envoi de TEMPERATURE, HUMIDITY et GET_DATE ; sinon, l'acquisition locale se poursuit sans republier ces informations.

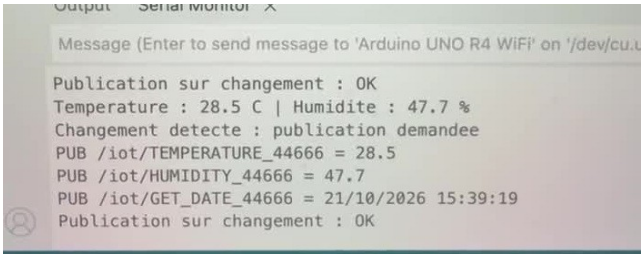


Figure 13 - Première branche du test : mesure à 28,5 °C et 47,7 %, puis publication des trois informations.

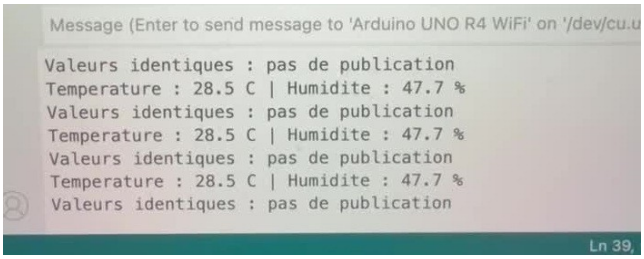


Figure 14 - Deuxième branche du test : le journal signale « Valeurs identiques : pas de publication ».

Analyse du comportement

Les deux états sont visibles dans la vidéo de la version optimisée. Le premier montre une publication effective ; le second indique plusieurs acquisitions identiques sans nouvelle publication de mesure. La gestion du LCD, de la LED et des messages entrants doit continuer dans les deux cas.

La comparaison porte sur les valeurs arrondies à une décimale. La référence est mise à jour uniquement lorsque les trois appels de publication réussissent. La version reste publiée après une commande 10 et une synchronisation NTP réussie.

Code 17 - Détection d'un changement à publier

```
if (premierePublication ||
    temperature != dernieretemperaturePubliee ||
    humidite != derniereHumiditePubliee) {
  publicationMesuresEnAttente = true;
} else {
  publicationMesuresEnAttente = false;
}
```

Le drapeau sépare l'acquisition de l'émission MQTT. Les dernières valeurs publiées sont conservées tant que l'envoi n'a pas abouti ; les messages série de diagnostic sont omis dans cet extrait.

6.1.6 Ordonnancement et suivi des publications

La comparaison des mesures est séparée de leur transmission. Cette organisation laisse les fonctions de réseau et de synchronisation s'exécuter à chaque passage dans la boucle. La mémorisation des dernières valeurs dépend ensuite du résultat des appels de publication.

Code 18 - Acquisition périodique et appels des services

```
unsigned long maintenant = millis();
if (mesuresActives &&
    maintenant - derniereMesure >= 2000UL) {
  dernieremesure = maintenant;
  lireDHT22();
}
gererWiFi();
gererMQTT();
synchroniserNTP();
publierVersion();
publierMesures();
```

La condition de deux secondes ne concerne que l'acquisition. Les appels de gestion réseau et de publication sont placés en dehors de cette condition. Le programme vérifie ainsi régulièrement les commandes entrantes, même lorsqu'aucune nouvelle mesure n'est encore due. Les appels réseau peuvent néanmoins introduire leurs propres délais.

Code 19 - Mémorisation des valeurs après le succès des trois appels

```
if (okTemp && okHum && okDate) {
  dernieretemperaturePubliee = temperature;
  dernieremiditePubliee = humidite;
  premierePublication = false;
  publicationMesuresEnAttente = false;
  Serial.println("Publication sur changement : OK");
}
```

La référence reste inchangée si au moins un appel retourne un échec. Une transmission partielle peut donc conduire à un nouvel envoi d'informations déjà émises. Le journal signale ici le succès des appels de la bibliothèque ; en QoS 0, ce message ne prouve pas un acquittement MQTT du broker ni le traitement par une IHM.

6.2.1 Analyse du dialogue MQTT avec Wireshark

L'analyse est menée sur le trafic MQTT vers le port TCP 1883. Le fichier 111.pcapng réalisé pendant la séance contient les phases de connexion, d'abonnement, de publication et de déconnexion. Une capture complémentaire a permis d'observer les messages de maintien de connexion avec un Keep Alive de 10 secondes.

Phase	Messages observés	Interprétation
Connexion	CONNECT / CONNACK	Le client ouvre la session ; le broker répond.
Abonnement	SUBSCRIBE / SUBACK	Le client demande un filtre ; le broker acquitte.
Publication	PUBLISH	Le message transporte un topic et une charge utile.
Maintien de connexion	PINGREQ / PINGRESP	Le client et le broker vérifient que la liaison reste active.
Déconnexion	DISCONNECT	Le client ferme proprement sa session MQTT.

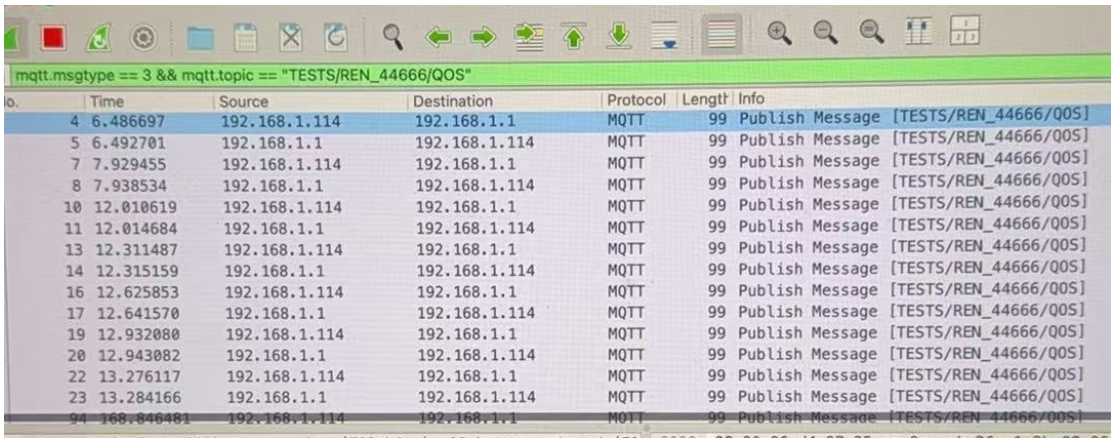


Figure 15 - Vue Wireshark des publications sur TESTS/REN_44666/QOS.

Lecture de la capture

La vue illustrée relie le poste 192.168.1.114 au broker 192.168.1.1 sur l'interface WiFi en0. Le filtre mqtt.msgtype == 3 sélectionne les PUBLISH. Pour suivre un dialogue complet, il faut conserver les autres types MQTT, car ce filtre masque les acquittements. L'adresse du poste de capture est distincte de celle du boîtier, 192.168.1.108.

6.2.2 Messages conservés avec Retain

Cette série d'essais étudie le comportement du broker à l'aide de topics de test propres au poste. Le premier test utilise TESTS/REN_44666/RETAIN. Le texte « Bonjour, test retain REN 44666 » est publié avec Retain activé.

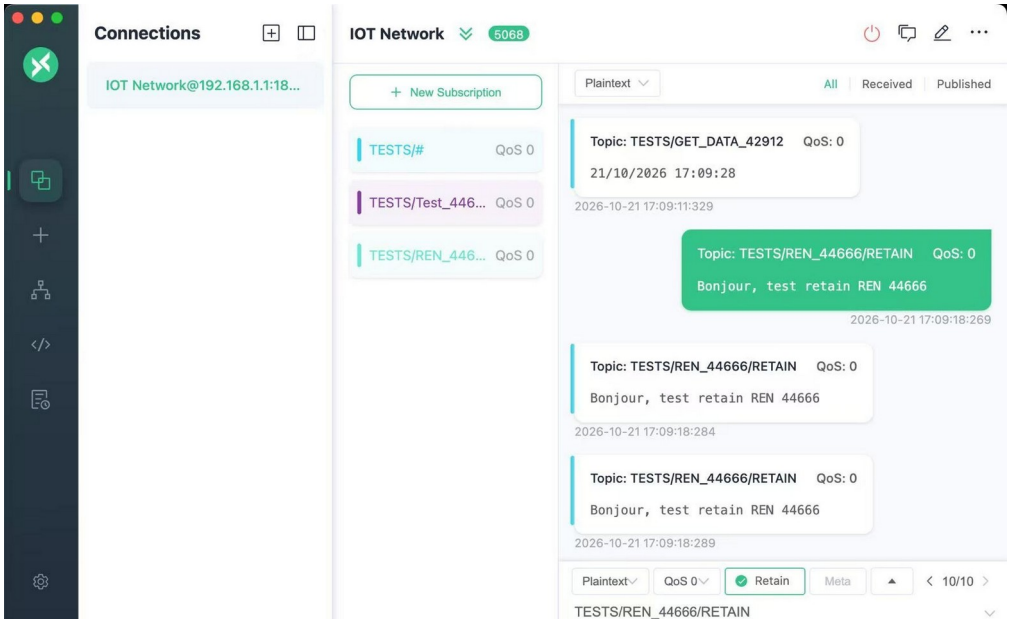


Figure 16 - Publication du message de test avec Retain activé dans MQTTX.

Étape de l'essai	Résultat validé pendant la séance
Publier un message avec Retain	Le broker conserve la dernière valeur sur ce topic.
Créer un nouvel abonnement	Le nouveau souscripteur reçoit la valeur conservée sans nouvelle publication.
Publier un contenu vide avec Retain	La valeur conservée est supprimée.
S'abonner à nouveau	L'ancien message n'est plus reçu.

Le test du nouvel abonnement distingue un message conservé d'une publication reçue en direct. Le test de suppression vérifie que le broker ne restitue plus l'ancienne valeur. Ces deux comportements ont été validés dans les essais du 21 octobre. Retain peut fournir la dernière mesure à une IHM qui se connecte plus tard ; l'horodatage permet alors d'en apprécier l'ancienneté.

6.2.2 Comparaison des niveaux QoS

Les niveaux QoS sont comparés sur le topic TESTS/REN_44666/QOS. La séance a permis de vérifier les trois procédures. Le niveau choisi concerne chaque liaison MQTT entre un émetteur et son destinataire ; le broker traite séparément la publication entrante et son transfert à un abonné.

QoS	Échange vérifié	Signification
0	PUBLISH	Au plus une fois ; pas d'acquittement MQTT.
1	PUBLISH puis PUBACK	Au moins une fois ; une retransmission peut créer un doublon.
2	PUBLISH, PUBREC, PUBREL, PUBCOMP	Exactement une fois dans l'échange MQTT concerné.

QoS 0

Une publication ne provoque pas d'acquittement MQTT de type PUBACK. Les acquittements TCP restent présents au niveau transport. Les publications de mesure reçues du boîtier dans MQTTX sont affichées avec QoS 0 ; elles doivent être distinguées des tests manuels de QoS 1 et de QoS 2.

QoS 1

Le PUBLISH et le PUBACK utilisent le même identifiant de paquet sur la liaison étudiée. Cette paire a été vérifiée pendant la séance. En l'absence d'acquittement, une nouvelle transmission peut être nécessaire, ce qui explique la garantie « au moins une fois ». Un filtre limité aux PUBLISH ne permet pas de voir le PUBACK.

QoS 2

La publication est suivie de PUBREC, PUBREL et PUBCOMP. La capture détaillée à la page suivante permet de suivre les identifiants dans les deux sens. Cette validation porte sur MQTTX et le broker ; elle ne signifie pas que le programme Arduino émet ses mesures en QoS 2.

Les résultats QoS 0 et QoS 1 sont repris des validations de la séance ; l'illustration détaillée conservée dans les documents fournis montre le dialogue QoS 2.

6.2.2 Analyse détaillée du dialogue QoS 2

La vue Wireshark élargit le filtre aux quatre types de messages du dialogue QoS 2. Deux suites sont visibles : celle du poste vers le broker et celle du broker vers le poste abonné. Les identifiants sont propres à chaque liaison.

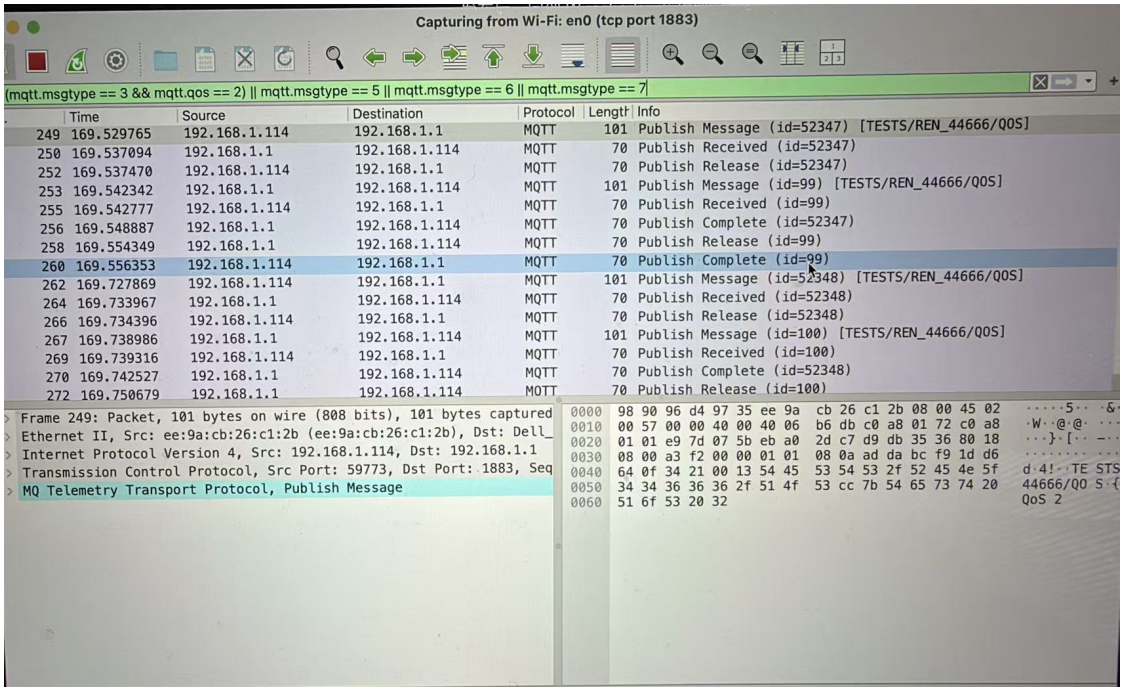


Figure 17 - Dialogue QoS 2 complet : PUBLISH, PUBREC, PUBREL et PUBCOMP.

Identifiant	Liaison	Suite des numéros de trame
52347	Poste vers broker	249 PUBLISH ; 250 PUBREC ; 252 PUBREL ; 256 PUBCOMP.
99	Broker vers abonné	253 PUBLISH ; 255 PUBREC ; 258 PUBREL ; 260 PUBCOMP.

Résultat : les quatre étapes sont présentes et peuvent être reliées par leur identifiant. La différence entre 52347 et 99 est normale : le broker termine un échange entrant puis conduit un échange distinct vers l'abonné. Le deuxième PUBLISH visible ne suffit donc pas à conclure à un doublon anormal.

6.2.2 Filtres de topics et jokers

Les publications de test permettent de comparer les abonnements exacts, le joker + et le joker #. Deux chemins de profondeurs différentes sont utilisés : TESTS/REN_44666/HUMIDITY avec la valeur 60, puis TESTS/REN_44666/SENSOR/ROOM1 avec le texte OK. Ces contenus sont saisis manuellement pour l'essai.

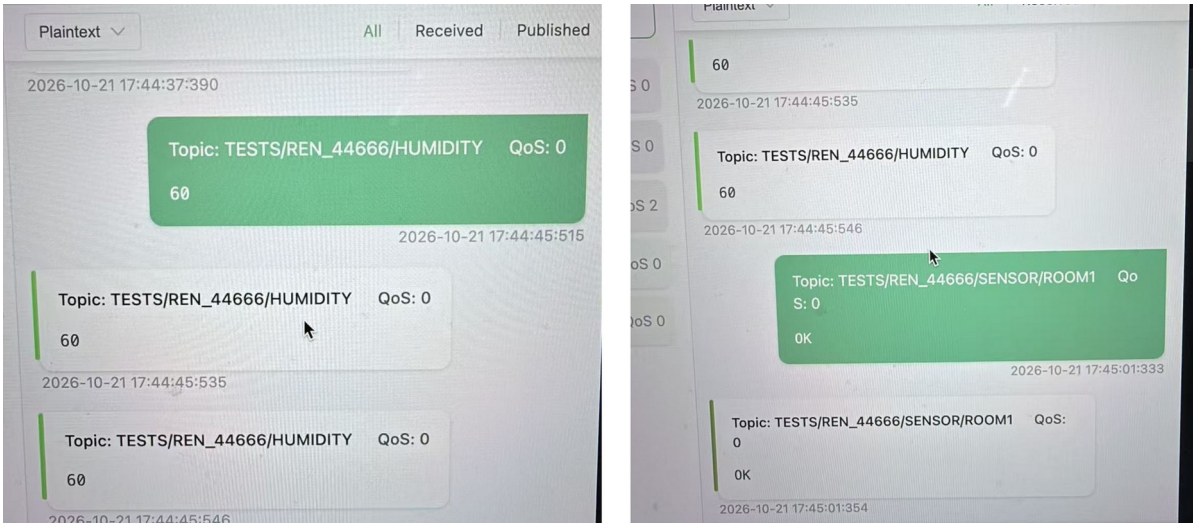


Figure 18 - Essais de réception sur HUMIDITY et SENSOR/ROOM1 dans MQTTX.

Filtre	Résultat dans cette arborescence
TESTS/REN_44666/HUMIDITY	Sélection du topic exact HUMIDITY.
TESTS/REN_44666/+	HUMIDITY correspond ; SENSOR/ROOM1 comporte un niveau supplémentaire.
TESTS/REN_44666/SENSOR/+	Sélection de ROOM1 au niveau situé après SENSOR.
TESTS/REN_44666/#	Sélection de toute cette branche et de ses sous-niveaux.

Résultat : + remplace exactement un niveau de topic, tandis que # couvre plusieurs niveaux. Les essais ont confirmé cette différence. L'utilisation d'un seul abonnement facilite l'analyse d'une règle ; plusieurs filtres correspondants rendent l'origine des réceptions moins facile à interpréter.

6.2.2 Supervision du broker avec SYS

L'abonnement explicite $SYS/# donne accès aux statistiques de fonctionnement de Mosquitto. Le broker publie notamment des compteurs d'octets et de messages, sa durée de fonctionnement et des indicateurs de charge. Ces informations décrivent le serveur partagé et ne sont pas des mesures du DHT22.

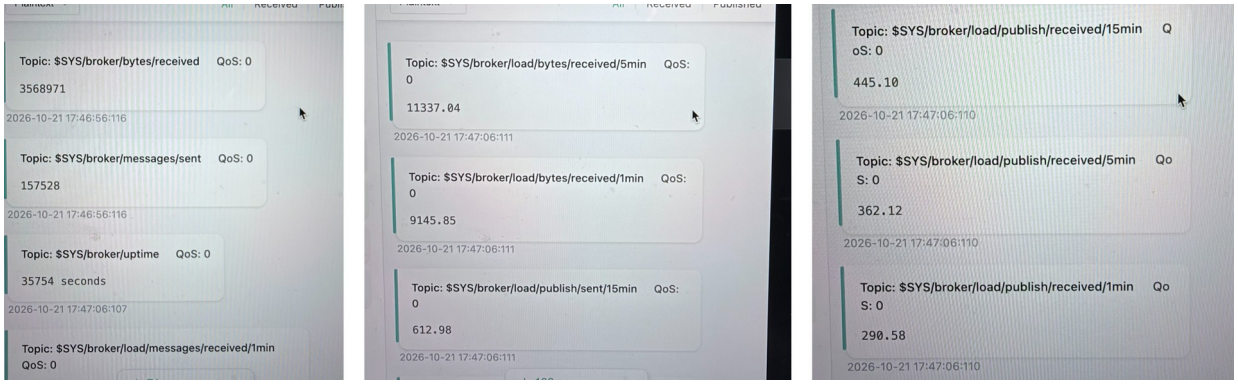


Figure 19 - Réception des indicateurs système du broker par l'abonnement $SYS/#.

Topic observé	Valeur	Lecture
$SYS/broker/bytes/received	356971	Octets reçus cumulés.
$SYS/broker/messages/sent	157528	Messages envoyés cumulés.
$SYS/broker/uptime	35754 seconds	Durée de fonctionnement.
$SYS/broker/load/bytes/received/1min	9145.85	Indicateur de charge sur une minute.
$SYS/broker/load/publish/received/5min	362.12	Indicateur de charge sur cinq minutes.

Résultat : les messages système sont bien reçus. Un abonnement ordinaire # ne sélectionne pas les topics commençant par $; l'abonnement explicite est nécessaire. Les valeurs ont été relevées à des instants différents et concernent tous les clients du broker : elles ne constituent ni une série chronologique homogène ni une mesure du trafic du seul Arduino.

IHM 3 Préparation et communication MQTT

Cette partie présente l'IHM réalisée le 3 novembre 2026 selon les étapes 3 à 6 du TP. Elle complète le boîtier 44666 par la lecture MQTT, les commandes, l'archivage SQLite et les courbes. Le programme final est lancé depuis le dossier IHM avec python main.py, dans l'environnement où paho-mqtt, PySide6 et matplotlib sont installés.

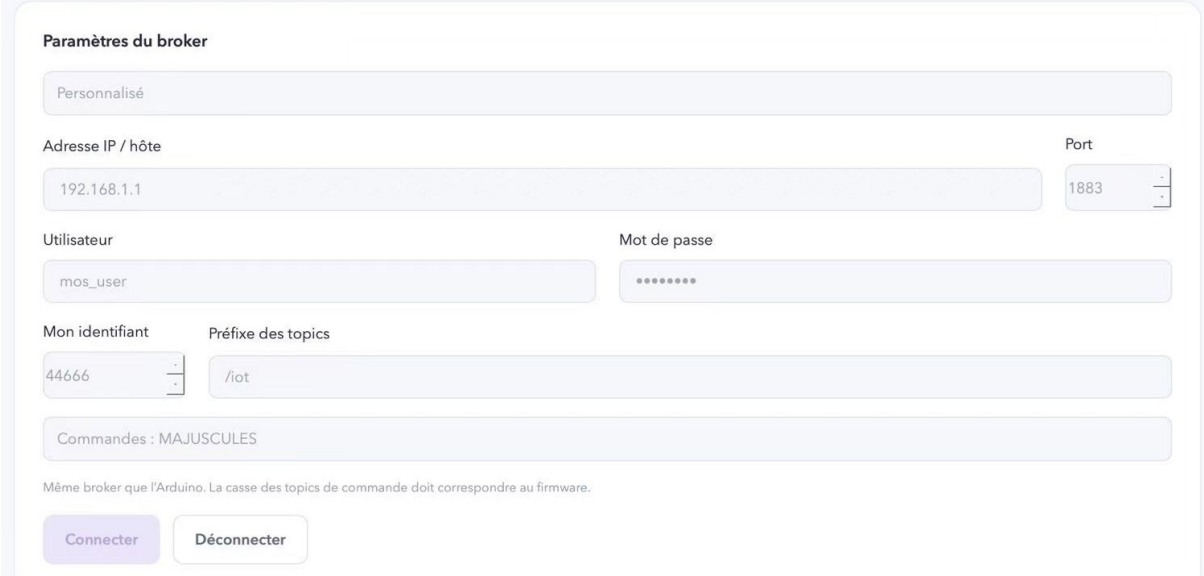


Figure 20 - Paramètres de connexion du programme : broker 192.168.1.1, port 1883, capteur 44666 et topics en majuscules.

Notice de démarrage : ouvrir Connexion, vérifier le broker et le port, conserver le préfixe /iot et le capteur 44666, puis cliquer sur Connecter. Le badge EN LIGNE et la barre d'état confirment la session. Cliquer sur Déconnecter avant de modifier les paramètres. Le broker doit être accessible depuis le réseau du poste.

3.2 à 3.5 Lecture et écriture en console

Les deux programmes console réutilisent ClientMQTT : connect_mqtt, disconnect_mqtt, publish_mqtt, subscribe_mqtt et wait_mqtt. Le lecteur utilise Client_L44666 et l'émetteur Client_E44666 ; ces identifiants distincts permettent leur exécution simultanée. Le menu d'écriture propose les commandes et le bargraphe.

Code 20 - Souscription aux quatre informations puis attente du lecteur

```
for name in subscriptions(cfg['prefix'], id_pc, cfg['uppercase']):
    mqtt.subscribe_mqtt(name)
mqtt.connect_mqtt()
mqtt.wait_mqtt(timeout=0)
```

Le lecteur conserve une boucle réseau permanente. Dans l'émetteur, loop_start() entretient MQTT pendant la saisie du menu. La casse configurée correspond aux topics Arduino /iot/TEMPERATURE_44666, /iot/HUMIDITY_44666, /iot/GET_DATE_44666 et /iot/VERSION_44666.

IHM 3.6 et 4 Acquisition et interface graphique

La fenêtre PySide6 sépare la vue d'ensemble, les commandes, la connexion, le journal et l'historique. Les fichiers mon_ihm.ui et graphe.ui sont conservés avec leurs versions Python ; python regenerate_ui.py relance la conversion avec pyside6-uic. Le fichier style.qss organise la présentation sans modifier le protocole du boîtier.

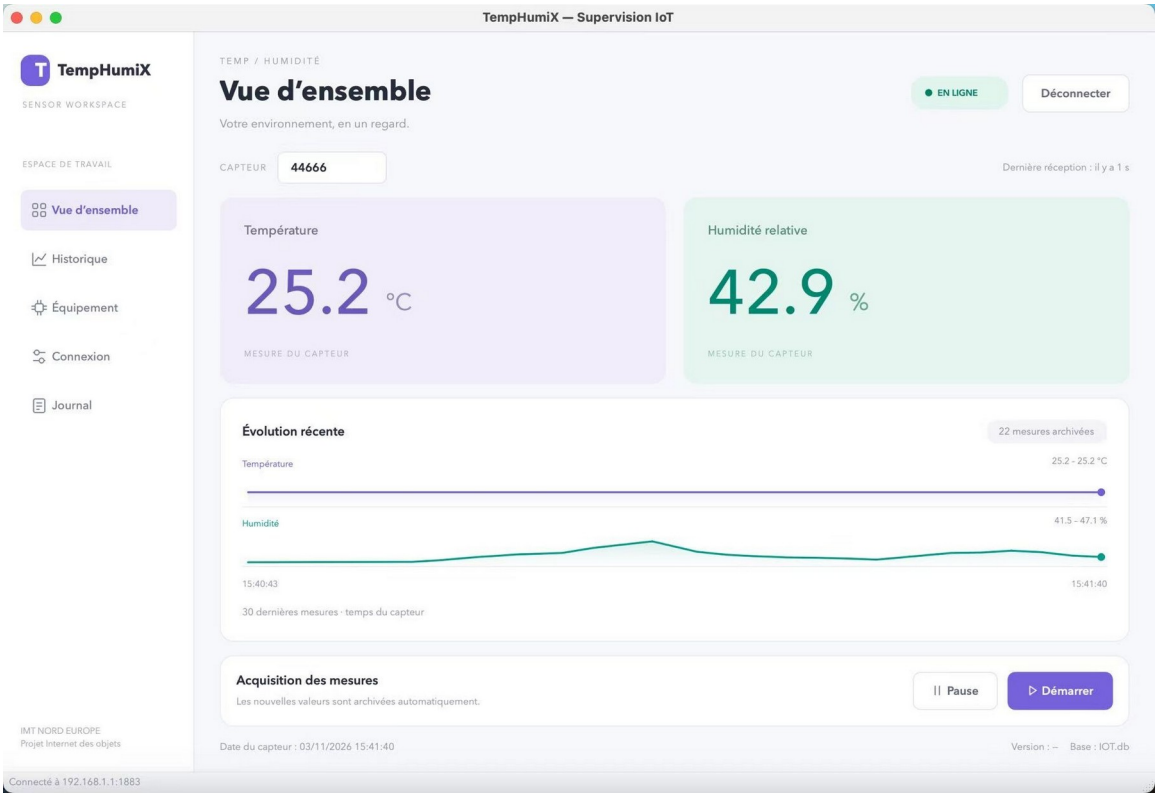


Figure 21 - Vue d'ensemble en ligne : 25,2 °C, 42,9 % d'humidité et 22 mesures archivées au moment de la capture.

Code 21 - Démarrage du thread de lecture et du rafraîchissement

```
self.mqtt.database_name = self.database
self.thread_mqtt = ThreadMQTT(self.mqtt)
self.timer.start(1000)
self.thread_mqtt.start()
```

ThreadMQTT, fondé sur threading.Thread, traite le réseau et l'écriture SQLite. Il transmet les événements par une Queue ; les widgets restent mis à jour dans le thread Qt. Un QTimer rafraîchit les valeurs chaque seconde. Démarrer lance les mesures, Pause les arrête ; la dernière valeur peut rester affichée. La courbe récente reprend au maximum les 30 dernières lignes archivées.

IHM 4 Commande du capteur

La page Équipement envoie les consignes au même capteur que celui de la connexion. Les champs contrôlent les bornes du bargraphe et de la période. L'aperçu des LED représente la valeur saisie ; l'exécution physique doit être observée sur le boîtier.

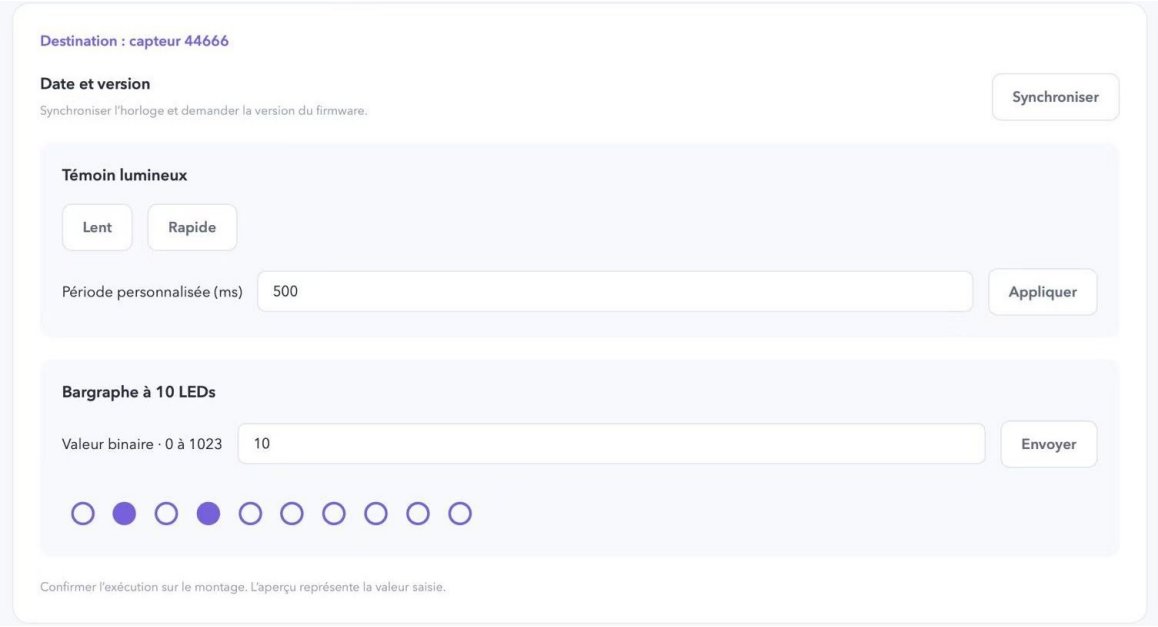


Figure 22 - Page Équipement : synchronisation, clignotement personnalisé et masque de bargraphe égal à 10.

Action dans l'IHM	Messages publiés
Synchroniser	COMMAND = 10
Démarrer / Pause	COMMAND = 20 puis 50 / COMMAND = 30 puis 40
Lent / Rapide	COMMAND = 40 / COMMAND = 50
Appliquer une période	BLINK = valeur, puis COMMAND = 60
Envoyer le bargraphe	BARGRAPH = valeur entière de 0 à 1023

Code 22 - Lecture et transmission de la période personnalisée

```
def customBlink(self):
    try:
        self.send_action('blink', int(self.lneBlink.text()))
    except ValueError:
        self.log('ERREUR : saisir une période entière')
```

Le module de commandes vérifie une période de 100 à 60 000 ms, par pas de 100 ms. Il publie la valeur BLINK avant le code 60. Le thread envoie les commandes avec QoS 1. L'acquittement du broker confirme l'échange MQTT, pas l'état matériel de la LED ou du bargraphe.

IHM 5 Réception et archivage SQLite

L'archivage associe les trois informations de mesure avant l'insertion. Sur ce montage, l'Arduino publie la température, l'humidité puis GET_DATE. Le programme attend donc un cycle complet, au lieu d'insérer systématiquement à la réception de HUMIDITY comme dans l'exemple du support.



Figure 23 - Détail du journal : réception des topics de température, d'humidité et de date du capteur 44666.

Id_capteur	Date	Version	Temperature	Humidite
INTEGER	TEXT en format ISO	TEXT	REAL en °C	REAL en %

Code 23 - Assemblage d'un cycle de mesure avant archivage

```
if all(key in self._pending for key in ('date', 'temperature', 'humidity')):
    row = dict(self._pending, sensor_id=self.id_capteur, version=self.version)
    self._pending.clear()
```

Code 24 - Insertion paramétrée dans IOT_DATA

```
cursor = self.conn.execute(
    'INSERT OR IGNORE INTO IOT_DATA VALUES (?,?,?,?,?)',
    (row['sensor_id'], row['date'], row['version'],
     row['temperature'], row['humidity']))
self.conn.commit()
```

Les dates du capteur sont normalisées en AAAA-MM-JJ HH:MM:SS. Un index unique sur capteur, date, température et humidité évite l'insertion de doublons identiques. La connexion d'écriture est ouverte et fermée dans le thread MQTT ; l'IHM utilise une connexion de lecture séparée. Les données réelles vont dans IOT.db et la démonstration dans IOT_demo.db.

La réception de trois topics ne constitue pas un identifiant de cycle partagé : un message manquant peut encore décaler l'association. Le journal distingue aussi l'heure du PC de celle transmise par le capteur ; leur différence ne mesure pas directement une latence réseau.

IHM 6 Consultation des courbes historiques

La fenêtre Historique relit les mesures stockées. L'utilisateur choisit le capteur, la date de début et la date de fin, puis clique sur Charger les données. Effacer les courbes nettoie uniquement l'affichage ; Fermer revient à la fenêtre principale. Une période vide ou inversée est signalée dans la fenêtre.

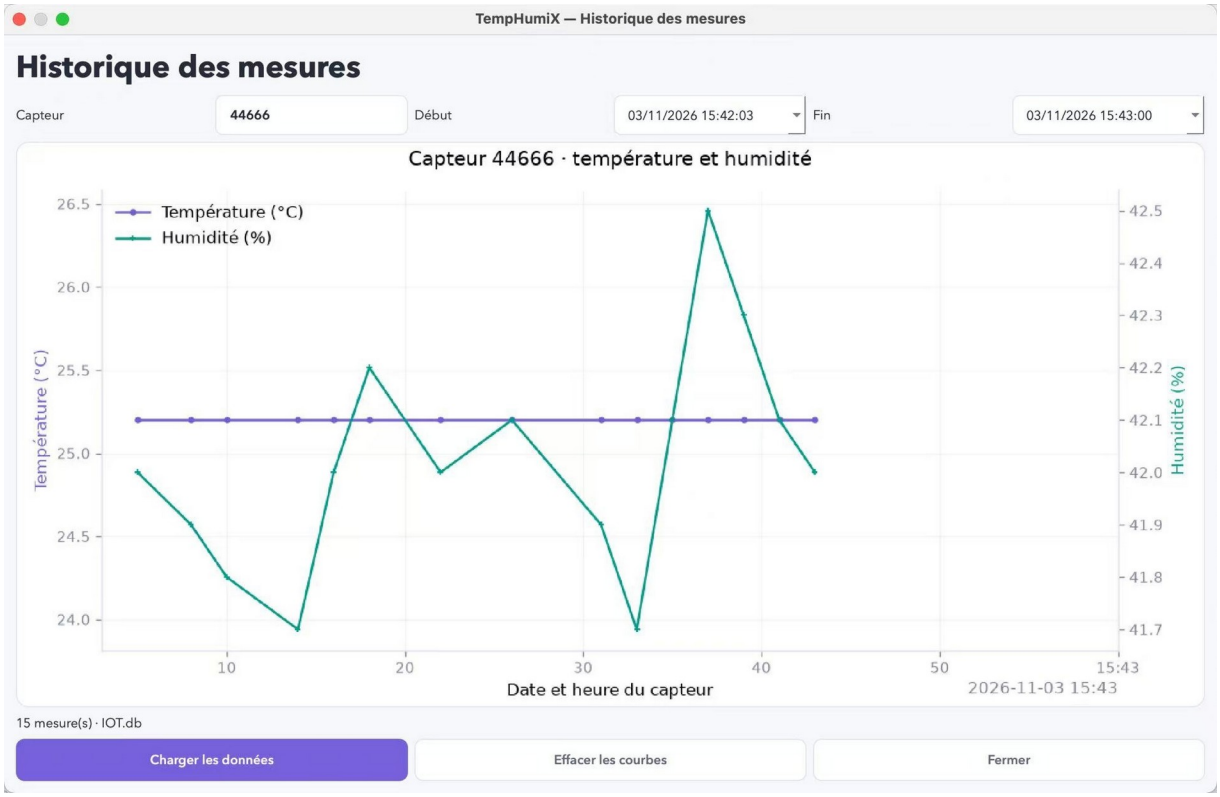


Figure 24 - Historique du capteur 44666 : 15 mesures sélectionnées entre 15:42:03 et 15:43:00 le 3 novembre 2026.

Code 25 - Lecture de l'intervalle et tracé des deux grandeurs

```
rows = db.query(int(self.cmbSensor.text()), start, end)
right = ax.twinx()
line_t, = ax.plot(dates, [r[1] for r in rows], color='#7061cf',
                  marker='o', markersize=3, label='Température (°C)')
line_h, = right.plot(dates, [r[2] for r in rows], color='#31998c',
                     marker='+', markersize=4, label='Humidité (%)')
```

La requête filtre Id_capteur et Date BETWEEN début AND fin, puis trie les lignes par date croissante. Les dates sont converties en objets datetime pour Matplotlib. La température utilise l'axe gauche et l'humidité l'axe droit, chacun avec son unité. La figure est rendue par FigureCanvasAgg, convertie en QPixmap puis placée dans une QGraphicsScene.

Sur l'intervalle présenté, la température reste à environ 25,2 °C tandis que l'humidité varie d'environ 41,7 à 42,5 %. Le compteur de 15 mesures concerne cet intervalle ; les 22 mesures de la vue d'ensemble correspondent à un autre instant et à l'ensemble déjà archivé.

IHM Bilan pratique et améliorations

Fonctions réalisées selon les étapes du TP

Étape du support	Réalisation et élément de vérification
3.2 à 3.5	ClientMQTT réutilisable ; lecteur et émetteur console à identifiants distincts.
3.6 et 4	ThreadMQTT, Queue et QTimer ; affichage en ligne de 25,2 °C et 42,9 %.
4.5	Commandes de mesure, synchronisation, clignotement et bargraphe ; page Équipement.
5	Table IOT_DATA, assemblage des messages et compteur de mesures archivées.
6	Sélection d'un intervalle et restitution de 15 mesures dans l'historique.

Points délicats rencontrés dans cette réalisation

Les exemples du support utilisent des topics en minuscules, alors que le firmware du montage publie en majuscules. Le réglage commun du préfixe et de la casse évite une connexion réussie sans aucune donnée visible. Le deuxième point est l'ordre des messages : GET_DATE arrive après les mesures, ce qui justifie l'assemblage du cycle avant l'écriture SQLite.

Le journal montre également un décalage entre l'heure du PC et celle du capteur. L'historique utilise l'heure du capteur ; un intervalle choisi sur la seule horloge du PC peut donc manquer les dernières mesures. Enfin, la vue d'ensemble affiche Version : — à l'instant photographié. Il faut demander la synchronisation pour déclencher la publication de VERSION prévue par le firmware.

Améliorations concrètes à poursuivre

Conserver dans SQLite deux dates distinctes, celle du capteur et celle de réception, faciliterait le diagnostic des décalages. Ajouter une durée maximale pour un cycle incomplet éviterait d'associer des valeurs trop éloignées dans le temps. Un export CSV de l'intervalle sélectionné permettrait de joindre facilement des résultats au compte rendu.

La reconnexion et l'arrêt du thread sont prévus dans le code. Un essai documenté de coupure du broker, puis de reprise, reste utile pour vérifier le comportement sur une longue séance. L'aperçu du bargraphe pourrait aussi être complété par un retour d'état du matériel afin de distinguer la consigne de l'action exécutée.

Acquis de cette partie

Cette réalisation m'a permis de relier une interface Qt à des échanges MQTT, de séparer le réseau de l'affichage et d'organiser une chaîne de données jusqu'à SQLite et Matplotlib. Les captures montrent la réception et l'historique ; le journal reste l'outil le plus direct pour comprendre une valeur absente ou un problème de topic.

Bilan des essais et versionnement

Partie	Résultat	Preuve utilisée
Simulation	Entrées et sorties intégrées.	Six captures SimulIDE, LCD et terminal.
6.1.1–6.1.2	Mesure locale et RTC.	Photographies du LCD.
6.1.3	Connexion WiFi et synchronisation.	Vidéo du démarrage et diagnostic de séance.
6.1.4	Réception des informations MQTT.	Capture MQTTX du capteur 44666.
6.1.5	Commande à distance des fonctions.	Vidéo MQTTX et activation du bargraphe.
6.1.6	Publication sur changement.	Deux états du journal dans la vidéo.
6.2.1	Dialogue et maintien de connexion.	Validation de 111.pcapng et capture complémentaire.
6.2.2	Retain, QoS, jokers et SYS.	Captures et résultats validés pendant la séance.
IHM 3 à 6	Supervision, commande et historique.	Cinq captures du programme et code Python.

Difficultés et corrections

La LED de vie ne réagissait pas avec l'affectation initiale ; le passage à D2 a résolu ce défaut. Les essais NTP ont ensuite nécessité un serveur accessible depuis le réseau du boîtier. Les diagnostics série et la séparation entre tests locaux et tests réseau ont permis de vérifier les corrections avant de poursuivre l'intégration.

Organisation du dépôt Git

Le dépôt iot_projet conserve les programmes et les circuits. Le répertoire simulide regroupe les montages simulés et arduino les programmes du boîtier. Le sketch final est rangé dans arduino/temphumix_r4/temphumix_r4.ino, avec un dossier portant le même nom que le fichier. Les commits successifs permettent de suivre l'ajout du clignotement, du chenillard, du DHT22, du LCD et de l'intégration TempHumiX.

Conclusion et ressources

Le projet a permis de réaliser un système de mesure connecté avec affichage local, horodatage et commande à distance. La progression sous SimulIDE a facilité la vérification des entrées et sorties avant le passage à l'UNO R4 WiFi. Les essais physiques ont ensuite validé l'intégration du capteur, de la RTC, du réseau et de MQTT.

L'optimisation des publications adapte les échanges aux variations des mesures. L'analyse avec MQTTX et Wireshark relie les actions du programme aux mécanismes de session, d'abonnement, d'acquittement et de conservation des messages. Les essais Retain, QoS, jokers et SYS complètent ainsi la validation fonctionnelle du boîtier.

L'IHM complète maintenant cette chaîne par une vue des mesures, des commandes à distance, un archivage SQLite et une consultation des courbes sur un intervalle choisi. Les prochaines améliorations portent sur le suivi des horloges, le traitement des cycles incomplets et l'export des résultats.

Ressources du projet

Dépôt GitHub du projet iot_projet

Supports de travaux pratiques IMT Nord Europe : simulation, communication MQTT et Construction de l'IHM. Essais documentés en octobre et le 3 novembre 2026.